

```

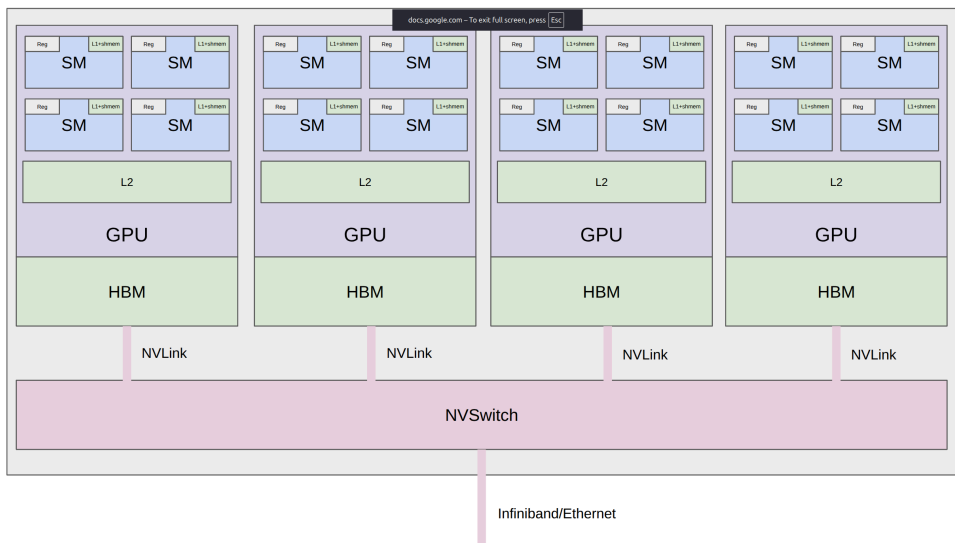
1 import torch
2 import time
3 import math
4 import sys
5 import os
6 from inspect import isfunction
7 from typing import Callable
8 from torch import nn, tensor
9 import torch.nn.functional as F
10 import torch.distributed as dist
11 import torch.multiprocessing as mp
12 from edtrace import text, image, link
13 from gpu_util import cuda_if_available
14 from lecture_util import article_link
15
16 if not torch.cuda.is_available():
17     torch.cuda.synchronize = lambda: None # No-op if CUDA is not available
18
19 def main():

```

Lecture 7: parallelism

Last week: parallelism within a single GPU

This week: parallelism across multiple GPUs



In both cases, **compute** (arithmetic logic units) is far from inputs/outputs (**data**).

Unifying theme: orchestrate computation to avoid data transfer bottlenecks

Generalized hierarchy:

- Single node, single GPU: L1 cache / shared memory (fastest)
- Single node, single GPU: HBM
- Single node, multi-GPU: NVLink/NVSwitch
- Multi-node, multi-GPU: Infiniband/Ethernet (slowest)

Last week: reduce memory accesses via fusion/tiling

This week: reduce communication across GPUs/nodes via replication/sharding

Why do multi-GPU?

1. Your parameters (optimizer state + gradients + activations) don't fit on a single GPU.
2. You want to use more GPUs (more FLOPs) to train faster.

When you execute this lecture directly (python lecture_07.py), it uses multiprocessing, which produces output from each process (below).

However, when you trace this lecture (python -m edtrace.execute -m lecture_07), we turn off multiprocessing.
[\[stdout for this lecture\]](#)

Part 1: building blocks of distributed communication/computation

```
collective_operations() # Programming model
hardware()              # Hardware: how GPUs are connected
torch_distributed()     # How this is implemented in NCCL/PyTorch
benchmarking()          # Measure actual NCCL bandwidth
```

Part 2: distributed training

Walk through bare-bones implementations of each strategy on deep MLPs.
Recall that MLPs are the compute bottleneck in Transformers, so this is representative.

```
data_parallelism()      # Cut up along the batch dimension
tensor_parallelism()    # Cut up along the width dimension
pipeline_parallelism()  # Cut up along the depth dimension
```

What's missing?

- Communication/computation overlap
- More general models (with attention, etc.)
- Other forms of parallelism (e.g., sequence parallelism, expert parallelism, combinations)
- Jax/TPUs: just define the model, the sharding strategy, and the Jax compiler handles the rest [\[levanter\]](#)
- But we're doing PyTorch so you can see how one builds up from the primitives

Summary

- Many ways to parallelize: data (batch), tensor/expert (width), pipeline (depth), sequence (length)
- Data parallelism: DDP (all-reduce), FSDP/ZeRO (all-gather + reduce-scatter)
- Tensor parallelism: requires very fast interconnects (e.g., NVLink)
- Pipeline parallelism: can work with slow interconnects, but need to work to reduce pipeline bubbles
- Can **re-compute** or store in **memory** or store in another GPUs memory and **communicate**
- Hardware is getting faster, but will always want bigger models, so will have this hierarchical structure

```
def collective_operations():
```

Collective operations are the conceptual primitives used for distributed programming [\[article\]](#)

- These are classic in the parallel programming literature from the 1980s.
- *Collective* means that you specify a general communication pattern across many devices.
- This can be better/faster than managing point-to-point communication yourself.

Setup:



- **Rank**: a particular device/GPU (e.g., 0, 1, 2, 3)
- **World size**: total number of devices (e.g., 4)

Operations:

- Broadcast, scatter, gather, reduce (foundations)

- All-gather, reduce-scatter, all-reduce (workhorse)
- All-to-all (for MoEs)

Broadcast: copy from rank 0 to all ranks

Input

```
rank0 = tensor([0., 1, 2, 3])
```

Output

```
rank0 = tensor([0., 1, 2, 3])
```

```
rank1 = tensor([0., 1, 2, 3])
```

```
rank2 = tensor([0., 1, 2, 3])
```

```
rank3 = tensor([0., 1, 2, 3])
```

Minor use case: rank 0 loads initial checkpoint and broadcasts to all ranks

Scatter tensor on rank 0 to all ranks

Input

```
rank0 = tensor([0., 1, 2, 3])
```

Output

```
rank0 = tensor([0.])
```

```
rank1 = tensor([1.])
```

```
rank2 = tensor([2.])
```

```
rank3 = tensor([3.])
```

Note: stepping stone to understanding reduce-scatter

Gather pieces from all ranks to rank 0 (opposite of scatter)

Input

```
rank0 = tensor([0.])
```

```
rank1 = tensor([1.])
```

```
rank2 = tensor([2.])
```

```
rank3 = tensor([3.])
```

Output

```
rank0 = tensor([0., 1, 2, 3])
```

Note: stepping stone to understanding all-gather

Reduce pieces from all ranks to rank 0, applying some operation (e.g., sum, min, max)

Input

```
rank0 = tensor([0.])
```

```
rank1 = tensor([1.])
```

```
rank2 = tensor([2.])
```

```
rank3 = tensor([3.])
```

Output

```
rank0 = tensor([6.]) # Sum of all ranks (0 + 1 + 2 + 3)
```

Note: stepping stone to understanding all-reduce

All-gather: perform gather to all ranks, not just rank 0

Input

```
rank0 = tensor([0.])
```

```
rank1 = tensor([1.])
```

```
rank2 = tensor([2.])
```

```
rank3 = tensor([3.])
```

```

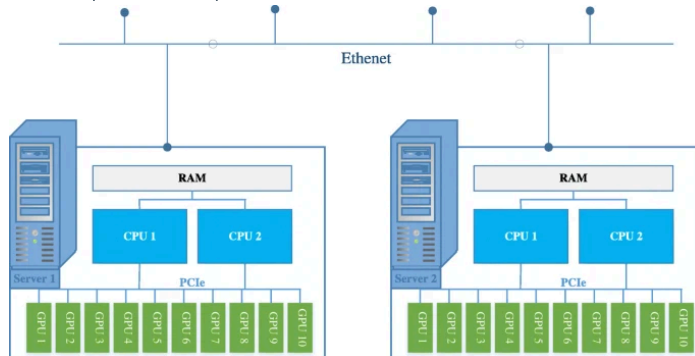
146 # Output
147 rank0 = tensor([0., 1, 2, 3])
148 rank1 = tensor([0., 1, 2, 3])
149 rank2 = tensor([0., 1, 2, 3])
150 rank3 = tensor([0., 1, 2, 3])
151
152 Use case: each rank holds parameter shard, gather to get full parameters for forward pass
153
154 Reduce-scatter: perform reduce on each dimension, scatter results
155 # Input
156 rank0 = tensor([0., 1, 2, 3])
157 rank1 = tensor([1., 2, 3, 4])
158 rank2 = tensor([2., 3, 4, 5])
159 rank3 = tensor([3., 4, 5, 6])
160
161 # Output
162 rank0 = tensor([6.]) # Sum along dim 0 (0 + 1 + 2 + 3)
163 rank1 = tensor([10.]) # Sum along dim 1 (1 + 2 + 3 + 4)
164 rank2 = tensor([14.]) # Sum along dim 2 (2 + 3 + 4 + 5)
165 rank3 = tensor([18.]) # Sum along dim 3 (3 + 4 + 5 + 6)
166
167 Use case: after backward pass, sum gradients from different data shards, but distribute storage
168
169 All-reduce = reduce-scatter + all-gather
170 # Input
171 rank0 = tensor([0., 1, 2, 3])
172 rank1 = tensor([1., 2, 3, 4])
173 rank2 = tensor([2., 3, 4, 5])
174 rank3 = tensor([3., 4, 5, 6])
175
176 # Output
177 rank0 = tensor([6., 10, 14, 18])
178 rank1 = tensor([6., 10, 14, 18])
179 rank2 = tensor([6., 10, 14, 18])
180 rank3 = tensor([6., 10, 14, 18])
181
182 Use case: after backward pass, sum gradients from different data shards, but replicate full parameters
183 Breaking all-reduce into reduce-scatter + all-gather allows for flexibility (e.g., ZeRO/FSDP)
184
185 All-to-all: each rank sends each other rank some tensor (most general)
186 # Input
187 rank0 = tensor([0., 1, 2, 3]) # send 0 to rank 0, 1 to rank 1, 2 to rank 2, 3 to rank 3
188 rank1 = tensor([4., 5, 6, 7]) # send 4 to rank 0, 5 to rank 1, 6 to rank 2, 7 to rank 3
189 rank2 = tensor([8., 9, 10, 11]) # send 8 to rank 0, 9 to rank 1, 10 to rank 2, 11 to rank 3
190 rank3 = tensor([12., 13, 14, 15]) # send 12 to rank 0, 13 to rank 1, 14 to rank 2, 15 to rank 3
191
192 # Output
193 rank0 = tensor([0, 4, 8, 12])
194 rank1 = tensor([1, 5, 9, 13])
195 rank2 = tensor([2, 6, 10, 14])
196 rank3 = tensor([3, 7, 11, 15])
197
198 Notes:
199 • Useful for MoEs: each rank has split of data and subset of experts; need to route data to experts
200 • For balanced splits, all-to-all looks like transpose
201 • Also handles unbalanced splits (but want splits to be as balanced as possible)
202
203 Way to remember the terminology:

```

- Reduce: performs some associative/commutative operation (sum, min, max)
- Scatter is inverse of gather
- All: means destination is all devices

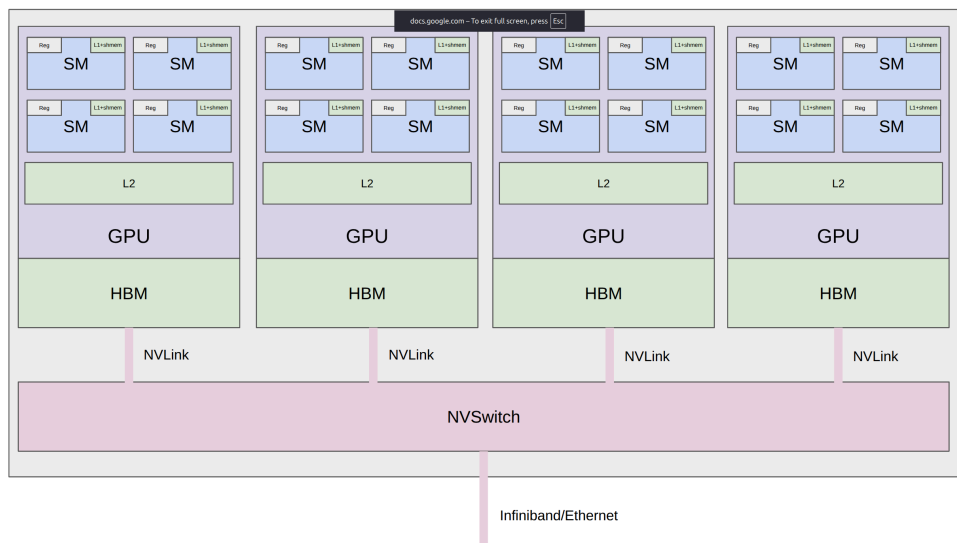
```
def hardware():
```

Classic (in the home):



- GPUs on same node communicate via a PCI(e) bus (v7.0, 16 lanes => 242 GB/s) [\[article\]](#)
- GPUs on different nodes communicate via Ethernet (~200 MB/s)

Modern (in the data center):



Typical setup:

- 8 GPUs per node, connected by NVLink to an NVSwitch (B200s' NVLink 5.0 gets 1.8 TB/s; HBM was 8 TB/s)
- 256 nodes per pod, connected by Infiniband (via PCIe -> HCA / Infiniband NIC -> Infiniband cable) (~0.05 TB/s)
- N pods per cluster / datacenter, connected by Ethernet (via PCIe -> CPU)

Bypassing the CPU:

- Ethernet requires passing through the CPU (copying data to kernel socket buffer, build TCP packets, copy to NIC ring buffer)
- Remote Direct Memory Access (RDMA): allows one GPU to directly read/write another GPU's memory without involving the CPU
- Infiniband supports RDMA, but standard Ethernet does not

Advancements:

- GB200/GB300 NVL72: 8 GPUs per tray, 9 trays per rack -> 72 GPUs in one NVLink domain
- RDMA over Converged Ethernet (RoCE): Ethernet bypasses CPU, similar but cheaper/weaker than Infiniband, used by Meta

232

NVIDIA Collective Communication Library (NCCL)

233 NCCL translates collective operations into low-level packets that are sent between GPUs. [\[talk\]](#)

234 • Detects topology of hardware (e.g., number of nodes, switches, NVLink/PCIe)

235 • Optimizes the path between GPUs

236 • Launches GPU kernels to send/receive data

237

238

239 `def torch_distributed():`

240 PyTorch distributed library (`torch.distributed`) [\[documentation\]](#)

241 • Provides clean interface for collective operations (e.g., `all_gather_into_tensor`)

242 • Supports multiple backends for different hardware: gloo (CPU), nccl (GPU)

243 • Also supports higher-level algorithms (e.g., `FullyShardedDataParallel`) [not used in this course]

244

245 Let's walk through some examples.

246 `spawn(collective_operations_main, world_size=4)`

247

248

249 `def collective_operations_main(rank: int, world_size: int):`

250 `"""This function is running asynchronously for each process (rank = 0, ..., world_size - 1)."""`

251 `setup(rank, world_size)`

252

253 `### All-reduce (dist = torch.distributed)`

254 `dist.barrier() # Waits for all processes to get to this point (in this case, for print statements)`

255

256 `data = tensor([0., 1, 2, 3], device=cuda_if_available(rank)) + rank # Both input and output`

257

258 `print(f"Rank {rank} [before all-reduce]: {data}", flush=True)`

259 `dist.all_reduce(tensor=data, op=dist.ReduceOp.SUM, async_op=False) # Modifies tensor in place`

260 `print(f"Rank {rank} [after all-reduce]: {data}", flush=True)`

261

262 `### Reduce-scatter`

263 `dist.barrier()`

264

265 `input = torch.arange(world_size, dtype=torch.float32, device=cuda_if_available(rank)) + rank # Input`

266 `output = torch.empty(1, device=cuda_if_available(rank)) # Allocate output`

267

268 `print(f"Rank {rank} [before reduce-scatter]: input = {input}, output = {output}", flush=True)`

269 `dist.reduce_scatter_tensor(output=output, input=input, op=dist.ReduceOp.SUM, async_op=False)`

270 `print(f"Rank {rank} [after reduce-scatter]: input = {input}, output = {output}", flush=True)`

271

272 `### All-gather`

273 `dist.barrier()`

274

275 `input = output # Input is the output of reduce-scatter`

276 `output = torch.empty(world_size, device=cuda_if_available(rank)) # Allocate output`

277

278 `print(f"Rank {rank} [before all-gather]: input = {input}, output = {output}", flush=True)`

279 `dist.all_gather_into_tensor(output_tensor=output, input_tensor=input, async_op=False)`

280 `print(f"Rank {rank} [after all-gather]: input = {input}, output = {output}", flush=True)`

281

282 Indeed, all-reduce = reduce-scatter + all-gather!

283

284 `cleanup()`

285

286

287 `def benchmarking():`

```

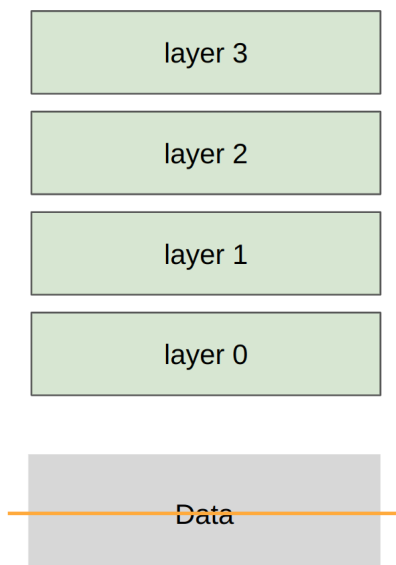
288     How fast does communication happen?
289
290     # All-reduce
291     spawn(all_reduce, world_size=4, num_elements=100 * 1024**2)
292
293     # Reduce-scatter
294     spawn(reduce_scatter, world_size=4, num_elements=100 * 1024**2)
295
296     References:
297     \[How to reason about collective operations\]
298     \[Sample benchmarking code\]
299
300
301     def all_reduce(rank: int, world_size: int, num_elements: int):
302         setup(rank, world_size)
303
304         # Create tensor
305         data = torch.randn(num_elements, device=cuda_if_available(rank))
306
307         # Warmup
308         dist.all_reduce(tensor=data, op=dist.ReduceOp.SUM, async_op=False)
309         torch.cuda.synchronize() # Wait for CUDA kernels to finish
310         dist.barrier()           # Wait for all the processes to get here
311
312         # Perform all-reduce
313         start_time = time.time()
314         dist.all_reduce(tensor=data, op=dist.ReduceOp.SUM, async_op=False)
315         torch.cuda.synchronize() # Wait for CUDA kernels to finish
316         dist.barrier()           # Wait for all the processes to get here
317         end_time = time.time()
318
319         duration = end_time - start_time
320         print(f"[all_reduce] Rank {rank}: all_reduce(world_size={world_size}, num_elements={num_elements}) took
{render_duration(duration)}", flush=True)
321
322         # Measure the effective bandwidth
323         dist.barrier()
324         size_bytes = data.element_size() * data.numel()
325         sent_bytes = size_bytes * 2 * (world_size - 1) # 2x because send + receive, world_size-1 steps in all-reduce
326         total_duration = world_size * duration
327         bandwidth = sent_bytes / total_duration
328         print(f"[all_reduce] Rank {rank}: all_reduce measured bandwidth = {round(bandwidth / 1024**3)} GB/s", flush=True)
329
330         # Notes:
331         # - Effective bandwidth ~ 2 * size_bytes / total_duration
332         # - Independent of world_size
333         # - Independent of topology (ring or tree)
334
335         cleanup()
336
337
338     def reduce_scatter(rank: int, world_size: int, num_elements: int):
339         setup(rank, world_size)
340
341         # Create input and outputs
342         input = torch.randn(world_size, num_elements, device=cuda_if_available(rank)) # Each rank has a matrix
343         output = torch.empty(num_elements, device=cuda_if_available(rank))
344

```

```

345     # Warmup
346     dist.reduce_scatter_tensor(output=output, input=input, op=dist.ReduceOp.SUM, async_op=False)
347     torch.cuda.synchronize() # Wait for CUDA kernels to finish
348     dist.barrier()           # Wait for all the processes to get here
349
350     # Perform reduce-scatter
351     start_time = time.time()
352     dist.reduce_scatter_tensor(output=output, input=input, op=dist.ReduceOp.SUM, async_op=False)
353     torch.cuda.synchronize() # Wait for CUDA kernels to finish
354     dist.barrier()           # Wait for all the processes to get here
355     end_time = time.time()
356
357     duration = end_time - start_time
358     print(f"[reduce_scatter] Rank {rank}: reduce_scatter(world_size={world_size}, num_elements={num_elements}) took
{render_duration(duration)}", flush=True)
359
360     # Measure the effective bandwidth
361     dist.barrier()
362     data_bytes = input.element_size() * input.numel() # How much data in the input
363     sent_bytes = data_bytes * (world_size - 1) # How much needs to be sent (no 2x here)
364     total_duration = world_size * duration # Total time for transmission
365     bandwidth = sent_bytes / total_duration
366     print(f"[reduce_scatter] Rank {rank}: reduce_scatter measured bandwidth = {round(bandwidth / 1024**3)} GB/s",
flush=True)
367
368     # Notes:
369     # - all-reduce = reduce-scatter + all-gather
370     # - all-reduce moves 2x the data in 2x the time compared to reduce-scatter, so similar bandwidth
371
372     cleanup()
373
374
375 def data_parallelism():
376

```



```

377     Sharding strategy: each rank gets a slice of the data
378
379     data = generate_sample_data()
380     spawn(data_parallelism_main, world_size=4, data=data, num_layers=4, num_steps=1)
381
382     Notes:
383     • Losses are different across ranks (computed on local data)

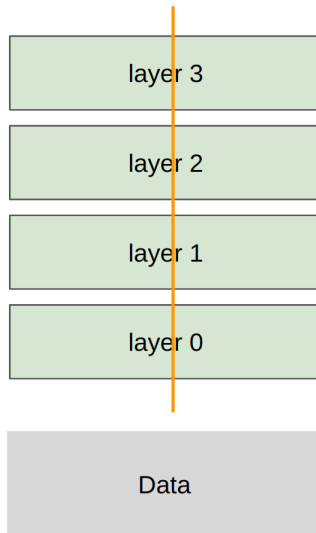
```


- Gradients are all-reduced to be the same across ranks
- Therefore, parameters remain the same across ranks

Next time: FSDP/ZeRO: use all-gather and reduce-scatter to avoid holding all parameters in memory

```
390 def generate_sample_data():
391     batch_size = 128
392     num_dim = 1024
393     data = torch.randn(batch_size, num_dim)
394     return data
395
396
397 def data_parallelism_main(rank: int, world_size: int, data: tensor, num_layers: int, num_steps: int):
398     setup(rank, world_size)
399
400     # Get the slice of data for this rank (in practice, each rank should load only its own data)
401     # --- B0 ---
402     # --- B1 ---
403     # --- B2 ---
404     # --- B3 ---
405     batch_size = data.size(0)
406     num_dim = data.size(1)
407     local_batch_size = int_divide(batch_size, world_size)
408     start_index = rank * local_batch_size
409     end_index = start_index + local_batch_size
410     data = data[start_index:end_index].to(cuda_if_available(rank))
411
412     # Create MLP parameters params[0], ..., params[num_layers - 1] (each rank has all parameters)
413     params = [get_init_params(num_dim, num_dim, rank) for layer in range(num_layers)]
414     optimizer = torch.optim.AdamW(params, lr=1e-3) # Each rank has own optimizer state
415
416     for step in range(num_steps):
417         # Forward pass
418         x = data
419         for param in params:
420             x = x @ param
421             x = F.gelu(x)
422         loss = x.square().mean() # Loss function is average squared magnitude
423
424         # Backward pass
425         loss.backward()
426
427         # Sync gradients across workers (ONLY difference between standard training and DDP)
428         for param in params:
429             dist.all_reduce(tensor=param.grad, op=dist.ReduceOp.AVG, async_op=False)
430
431         # Update parameters
432         optimizer.step()
433
434         print(f"[data_parallelism] Rank {rank}: step = {step}, loss = {loss.item()}, params =
435         {[summarize_tensor(params[layer]) for layer in range(num_layers)]}", flush=True)
436
437     cleanup()
438
439 def tensor_parallelism():
```

440



441 Sharding strategy: each rank gets part of each layer, transfer all data/activations

442

443 data = generate_sample_data()

444 spawn(tensor_parallelism_main, world_size=4, data=data, num_layers=4)

445

446

447 def tensor_parallelism_main(rank: int, world_size: int, data: tensor, num_layers: int):

448 setup(rank, world_size)

449

450 data = data.to(cuda_if_available(rank)) # All ranks get the data (batch_size x num_dim)

451 batch_size = data.size(0)

452 num_dim = data.size(1)

453 local_num_dim = int_divide(num_dim, world_size) # Shard `num\_dim`

454

455 # Create model (each rank gets 1/world_size of the parameters)

456 # | | | |

457 # W0 W1 W2 W3

458 # | | | |

459 params = [get_init_params(num_dim, local_num_dim, rank) for layer in range(num_layers)]

460

461 # Forward pass

462 x = data

463 for layer in range(num_layers):

464 # Compute activations (batch_size x local_num_dim)

465 x = x @ params[layer] # Note: this is only on a slice of the parameters

466 x = F.gelu(x)

467

468 # Allocate memory for activations (world_size x batch_size x local_num_dim)

469 activations = [torch.empty(batch_size, local_num_dim, device=cuda_if_available(rank)) for _ in range(world_size)]

470

471 # Send activations via all gather

472 dist.all_gather(tensor_list=activations, tensor=x, async_op=False)

473

474 # Concatenate them to get batch_size x num_dim

475 x = torch.cat(activations, dim=1)

476

477 print(f"[tensor_parallelism] Rank {rank}: forward pass produced activations {summarize_tensor(x)}", flush=True)

478

479 # Backward pass: homework exercise

480

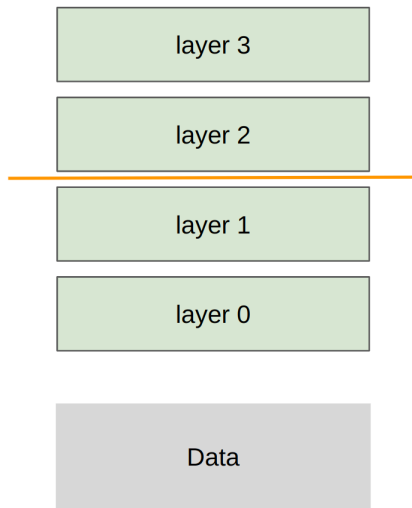
481 cleanup()

482

483

```
484 def pipeline_parallelism():
```

485



486 Sharding strategy: each rank gets subset of layers, transfer all data/activations

487

```
488 data = generate_sample_data()
```

```
489 spawn(pipeline_parallelism_main, world_size=2, data=data, num_layers=4, num_micro_batches=4)
```

490

491

```
492 def pipeline_parallelism_main(rank: int, world_size: int, data: tensor, num_layers: int, num_micro_batches: int):
```

```
493     setup(rank, world_size)
```

494

```
495     # Use all the data
```

```
496     data = data.to(cuda_if_available(rank))
```

```
497     batch_size = data.size(0)
```

```
498     num_dim = data.size(1)
```

499

```
500     # Split up layers
```

```
501     local_num_layers = int_divide(num_layers, world_size)
```

502

```
503     # Each rank gets a subset of layers
```

```
504     local_params = [get_init_params(num_dim, num_dim, rank) for layer in range(local_num_layers)]
```

505

```
506     # Forward pass
```

507

```
508     # Break up into micro batches to minimize the bubble
```

```
509     micro_batch_size = int_divide(batch_size, num_micro_batches)
```

```
510     if rank == 0:
```

```
511         # The data
```

```
512         micro_batches = data.chunk(chunks=num_micro_batches, dim=0)
```

```
513     else:
```

```
514         # Allocate memory for activations
```

```
515         micro_batches = [torch.empty(micro_batch_size, num_dim, device=cuda_if_available(rank)) for _ in
```

```
range(num_micro_batches)]
```

516

```
517     for x in micro_batches:
```

```
518         # Get activations from previous rank
```

```
519         if rank - 1 >= 0:
```

```
520             dist.recv(tensor=x, src=rank - 1)
```

521

```
522         # Compute layers assigned to this rank
```

```
523         for param in local_params:
```

```
524             x = x @ param
```

```

525         x = F.gelu(x)
526
527     # Send to the next rank
528     if rank + 1 < world_size:
529         print(f"[pipeline_parallelism] Rank {rank}: sending {summarize_tensor(x)} to rank {rank + 1}", flush=True)
530         dist.send(tensor=x, dst=rank + 1)
531
532     Not handled: overlapping communication/computation to eliminate pipeline bubbles
533
534     # Backward pass: homework exercise
535
536     cleanup()
537
538     #####
539
540 def setup(rank: int, world_size: int):
541     """Initializes the distributed environment (called at start of process)."""
542     # Specify where master lives (rank 0), used to coordinate (actual data goes through NCCL)
543     os.environ["MASTER_ADDR"] = "localhost"
544     os.environ["MASTER_PORT"] = "15623"
545
546     if torch.cuda.is_available():
547         dist.init_process_group("nccl", rank=rank, world_size=world_size)
548     else:
549         dist.init_process_group("gloo", rank=rank, world_size=world_size)
550
551
552 def cleanup():
553     """Cleans up the distributed environment (called at end of process)."""
554     torch.distributed.destroy_process_group()
555
556
557 class DisableDistributed:
558     """
559     Context manager that temporarily disables distributed functions (replaces with no-ops).
560     This is for when we're tracing the lecture, since we can't trace through
561     multiprocessing, so we just want to run the function directly without
562     distributed communication.
563     """
564     def __enter__(self):
565         self.old_functions = {}
566         for name in dir(dist):
567             value = getattr(dist, name, None)
568             if isfunction(value):
569                 self.old_functions[name] = value
570                 setattr(dist, name, lambda *args, **kwargs: None)
571
572     def __exit__(self, exc_type, exc_value, traceback):
573         for name in self.old_functions:
574             setattr(dist, name, self.old_functions[name])
575
576
577 def spawn(func: Callable, world_size: int, *args, **kwargs):
578     """
579     Launches `world_size` processes that each calls `func` on world_size, args, kwargs.
580     Note: if we are being traced (inside edtrace), we just run the function directly without multiprocessing and disable
581     distributed functions.
582     """

```

```

582     # Note: assume kwargs are in the same order as what main needs
583     if not sys.gettrace():
584         # This is the normal code path for multiprocessing
585         args = (world_size,) + args + tuple(kwargs.values())
586         mp.spawn(func, args=args, nprocs=world_size, join=True)
587     else:
588         # If we're being traced (inside edtrace), just run the function directly.
589         with DisableDistributed():
590             args = (0, world_size,) + args + tuple(kwargs.values())
591             func(*args)
592
593
594 def get_init_params(num_inputs: int, num_outputs: int, rank: int) -> nn.Parameter:
595     """Create parameters and put them on the `rank`-th GPU."""
596     torch.random.manual_seed(0) # For reproducibility
597     return nn.Parameter(torch.randn(num_inputs, num_outputs, device=cuda_if_available(rank)) / math.sqrt(num_outputs))
598
599
600 def int_divide(a: int, b: int):
601     """Return a / b and throw an error if there's a remainder."""
602     assert a % b == 0
603     return a // b
604
605
606 def summarize_tensor(tensor: tensor) -> str:
607     return "x".join(map(str, tensor.shape)) + "[" + str(round(tensor.view(-1)[0].item(), 4)) + "...]"
608
609
610 def render_duration(duration: float) -> str:
611     if duration < 1e-3:
612         return f"{duration * 1e6:.2f}us"
613     if duration < 1:
614         return f"{duration * 1e3:.2f}ms"
615     return f"{duration:.2f}s"
616
617
618 if __name__ == "__main__":
619     main()

```